



Object-Oriented Programming

Chapter 10

10.1 Introduction

- Classes are new data types
- Create your own *custom* classes
- Object-oriented programming makes it easier to design, implement, test, debug and update applications
- To take maximum advantage of Python you must familiarize yourself with lots of preexisting classes
- Easy for you to reuse existing classes rather than “reinventing the wheel”
- If you become part of a development team in industry, you may work on applications that contain hundreds, or even thousands, of classes

Inheritance

- When creating a new class, you can **inherit** the attributes (variables) and methods (the class version of functions) of a previously defined **base class** (also called a **superclass**)
- New class is called a **derived class** (or **subclass**)
- You then customize the derived class to meet the specific needs of your application

10.2 Custom Class Account

- Bank `Account` class that holds an account holder's name and balance
 - An actual bank account class would likely include lots of other information

```
# account.py
"""Account class definition."""
from decimal import Decimal

class Account:
    """Account class for maintaining a bank account balance."""

    def __init__(self, name, balance):
        """Initialize an Account object."""

        # if balance is less than 0.00, raise an exception
        if balance < Decimal('0.00'):
            raise ValueError('Initial balance must be >= to 0.00.')

        self.name = name
        self.balance = balance

    def deposit(self, amount):
        """Deposit money to the account."""

        # if amount is less than 0.00, raise an exception
        if amount < Decimal('0.00'):
            raise ValueError('amount must be positive.')

        self.balance += amount
```

Importing Classes `Account` and `Decimal`

- Launch IPython from the `ch10` examples folder, then import class `Account`

```
In [1]: from account import Account
```

- `Account` maintains and manipulates the account balance as a `Decimal`

```
In [2]: from decimal import Decimal
```

Create an `Account` Object with a Constructor Expression

- Create an object with a **constructor expression** that builds and initializes an object
- Constructor expressions create new objects and initialize their data using argument(s) specified in parentheses
- Parentheses following the class name are required, even if there are no arguments

```
In [3]: account1 = Account('John Green', Decimal('50.00'))
```

Getting an `Account`'s Name and Balance

- Access the `Account` object's `name` and `balance` attributes

```
In [4]: account1.name
```

```
Out[4]: 'John Green'
```

```
In [5]: account1.balance
```

```
Out[5]: Decimal('50.00')
```

Depositing Money into an Account

- `deposit` method receives a positive dollar amount and adds it to the balance

```
In [6]: account1.deposit(Decimal('25.53'))
```

```
In [7]: account1.balance
```

```
Out[7]: Decimal('75.53')
```

Account Methods Perform Validation

- `Account`'s methods validate their arguments

```
In [8]: account1.deposit(Decimal('-123.45'))
```

```
-----  
ValueError                                Traceback (most recent call last)  
<ipython-input-8-27dc468365a7> in <module>  
----> 1 account1.deposit(Decimal('-123.45'))  
  
~/Dropbox/books/2019/Python/PyCDS_JupyterSlides/ch10/account.py in deposit(self, amount)  
    21         # if amount is less than 0.00, raise an exception  
    22         if amount < Decimal('0.00'):  
--> 23             raise ValueError('amount must be positive.')  
    24  
    25         self.balance += amount
```

```
ValueError: amount must be positive.
```

10.2.2 Account Class Definition

Defining a Class

- Class definition begins with the keyword `class` followed by the class's name and a colon (`:`)
- Called the **class header**
- *Style Guide for Python Code* recommends that you begin each word in a multi-word class name with an uppercase letter
- Every statement in a class's suite is indented

```
# account.py
"""Account class definition."""
from decimal import Decimal

class Account:
    """Account class for maintaining a bank account balance."""
```

- Each class typically provides a descriptive docstring
- Must appear in the line or lines immediately following the class header
- View any class's docstring in IPython, type the class name and a question mark, then press *Enter*

In [9]: Account?

```
Init signature: Account(name, balance)
Docstring:      Account class for maintaining a bank account balance.
Init docstring: Initialize an Account object.
File:           ~/Dropbox/books/2019/PythonFullThrottle/account.py
Type:           type
Subclasses:
```

Initializing Account Objects: Method `__init__`

- Constructor expression creates a new object, then initializes its data by calling the class's `__init__` method
- Each new class can provide an `__init__` method that specifies how to initialize an object's data attributes
- Returning a value other than `None` from `__init__` results in a `TypeError`
- Class `Account`'s `__init__` method initializes an `Account` object's `name` and `balance` attributes if the `balance` is valid

Initializing Account Objects: Method `__init__` (cont.)

```
def __init__(self, name, balance):  
    """Initialize an Account object."""  
  
    # if balance is less than 0.00, raise an exception  
    if balance < Decimal('0.00'):  
        raise ValueError('Initial balance must be >= to 0.00.')  
  
    self.name = name  
    self.balance = balance
```

Initializing Account Objects: Method `__init__` (cont.)

- When you call a method for a specific object, Python implicitly passes a reference to that object as the method's first argument
- So all methods of a class must specify at least one parameter
- By convention a method's first parameter is named `self`
- Methods must use that reference (`self`) to access the object's attributes and other methods

Initializing Account Objects: Method `__init__` (cont.)

- When an object is created, it does not yet have any attributes
- They're added *dynamically* via assignments of the form:

```
self.attribute_name = value
```

Initializing Account Objects: Method `__init__` (cont.)

- Python classes may define many [special methods](#), like `__init__` [Click link](#)
- Each identified by leading and trailing double-underscores (`__`) in the method name
- Class `object` defines the special methods that are available for *all* Python objects

Method `deposit`

- Adds a positive `amount` to the account's `balance` attribute
- Raises a `ValueError` if `amount` is less than `0.00`

```
def deposit(self, amount):  
    """Deposit money to the account."""  
  
    # if amount is less than 0.00, raise an exception  
    if amount < Decimal('0.00'):  
        raise ValueError('amount must be positive.')  
  
    self.balance += amount
```

10.2.3 Composition: Object References as Members of Classes

- An Account *has a* `name`, and an `Account` *has a* `balance`
- An object's attributes are references to objects of other classes
- Embedding references to objects of other types is a form of software reusability known as **composition** and is sometimes referred to as the “**has a**” relationship

10.3 Controlling Access to Attributes

- Pprevious example used attributes `name` and `balance` only to *get* the values of those attributes
- Also can use those attributes to *modify* their values

```
In [1]: from account import Account
```

```
In [2]: from decimal import Decimal
```

```
In [3]: account1 = Account('John Green', Decimal('50.00'))
```

```
In [4]: account1.balance
```

```
Out[4]: Decimal('50.00')
```

- Set the `balance` attribute to an *invalid* negative value, then display the `balance`

```
In [5]: account1.balance = Decimal('-1000.00')
```

```
In [6]: account1.balance
```

```
Out[6]: Decimal('-1000.00')
```

Encapsulation

- A class's **client code** is any code that uses objects of the class
- Most object-oriented programming languages enable you to **encapsulate** (or *hide*) an object's data from the client code
 - *private data*

Leading Underscore (`_`) Naming Convention

- Python does *not* have private data
- Use *naming conventions* to design classes that encourage correct use
- By convention, Python programmers know that any attribute name beginning with an underscore (`_`) is for a class's *internal use only*
- Attributes whose identifiers do *not* begin with an underscore (`_`) are considered *publicly accessible* for use in client code

10.4 Time Class with Properties for Data Access

- **Properties** can control the manner in which they get and modify an object's data—**assuming programmers follow conventions**
- For robust date and time manipulation capabilities, see Python's [datetime module](#) [Click link](#)

10.4.1 Test-Driving Class Time

Before we look at class `Time`'s definition, let's demonstrate its capabilities

```
In [1]: from timewithproperties import Time
```

Creating a Time Object

- Create a `Time` object
- Class `Time`'s `__init__` method has `hour`, `minute` and `second` parameters, each with a default argument value of 0

```
In [2]: wake_up = Time(hour=6, minute=30)
```

```
# timewithproperties.py
"""Class Time with read-write properties."""

class Time:
    """Class Time with read-write properties."""

    def __init__(self, hour=0, minute=0, second=0):
        """Initialize each attribute."""
        self.hour = hour # 0-23
        self.minute = minute # 0-59
        self.second = second # 0-59
```

Displaying a Time Object

- Class `Time` defines two methods that produce string representations of `Time` object
- When you evaluate a variable in IPython, it calls the object's `__repr__` special method to produce a string representation of the object

```
In [3]: wake_up
```

```
Out[3]: Time(hour=6, minute=30, second=0)
```

- The `__str__` special method is called when an object is converted to a string, such as when you output the object with `print`

```
In [4]: print(wake_up)
```

```
6:30:00 AM
```

```
def __repr__(self):
    """Return Time string for repr()."""
    return (f'Time(hour={self.hour}, minute={self.minute}, ' +
            f'second={self.second})')

def __str__(self):
    """Return Time string in 12-hour clock format."""
    return ((f'12' if self.hour in (0, 12) else str(self.hour % 12)) +
            f':{self.minute:0>2}:{self.second:0>2}' +
            (' AM' if self.hour < 12 else ' PM'))
```

Getting an Attribute Via a Property

- Class `Time` provides `hour`, `minute` and `second` **properties**
 - Provide the convenience of data attributes for getting and modifying an object's data
 - Implemented as methods, so they may contain additional logic

```
In [5]: wake_up.hour
```

```
Out[5]: 6
```

- Appears to simply get an `hour` data attribute's value
- Actually a call to an `hour` *method* that returns the value of an `_hour` data attribute

```
@property
def hour(self):
    """Return the hour."""
    return self._hour
```

Setting the Time

- `Time` method `set_time` method provides `hour`, `minute` and `second` parameters, each with a default of `0`

```
In [6]: wake_up.set_time(hour=7, minute=45)
```

```
In [7]: wake_up
```

```
Out[7]: Time(hour=7, minute=45, second=0)
```

```
def set_time(self, hour=0, minute=0, second=0):  
    """Set values of hour, minute, and second."""  
    self.hour = hour  
    self.minute = minute  
    self.second = second
```

Setting an Attribute via a Property

- Class `Time` also supports setting the `hour`, `minute` and `second` values individually via its properties

```
In [8]: wake_up.hour = 6
```

```
In [9]: wake_up
```

```
Out[9]: Time(hour=6, minute=45, second=0)
```

- Appears to simply assign a value to a data attribute
- Actually a call to an `hour` method that takes `6` as an argument, validates the value, then assigns it to a corresponding data attribute named `_hour`

```
@hour.setter
def hour(self, hour):
    """Set the hour."""
    if not (0 <= hour < 24):
        raise ValueError(f'Hour ({hour}) must be 0-23')

    self._hour = hour
```

Attempting to Set an Invalid Value

To prove that class `Time`'s properties *validate* the values you assign to them, let's try to assign an invalid value to the `hour` property, which results in a `ValueError` :

```
In [10]: wake_up.hour = 100
```

```
-----  
ValueError                                Traceback (most recent call last)  
<ipython-input-10-1fce0716ef14> in <module>  
----> 1 wake_up.hour = 100  
  
~/Dropbox/books/2019/Python/PyCDS_JupyterSlides/ch10/timewithproperties.py in hour(self, hour)  
    20         """Set the hour."""  
    21         if not (0 <= hour < 24):  
----> 22             raise ValueError(f'Hour ({hour}) must be 0-23')  
    23  
    24         self._hour = hour  
  
ValueError: Hour (100) must be 0-23
```

10.4.2 Class Time Definition

Class Time: `__init__` Method with Default Parameter Values

- Specifies `hour`, `minute` and `second` parameters, each with a default argument of `0`
- The statements containing `self.hour`, `self.minute` and `self.second` *appear* to create `hour`, `minute` and `second` attributes for the new `Time` object (`self`)
- These statements actually call methods that implement the class's `hour`, `minute` and `second` *properties*
- Those methods create attributes named `_hour`, `_minute` and `_second`

```
# timewithproperties.py
"""Class Time with read-write properties."""

class Time:
    """Class Time with read-write properties."""

    def __init__(self, hour=0, minute=0, second=0):
        """Initialize each attribute."""
        self.hour = hour    # 0-23
        self.minute = minute # 0-59
        self.second = second # 0-59
```

Class Time: hour Read-Write Property

- Methods named `hour` define a *publicly accessible read-write property* named `hour` that manipulates a data attribute named `_hour`
- The single-leading-underscore (`_`) naming convention indicates that client code should not access `_hour` directly
- Properties look like data attributes to programmers working with `Time` objects, but are implemented as *methods*
- Each property defines a *getter* method which *gets* (returns) a data attribute's value
- Each property can *optionally* define a *setter* method which *sets* a data attribute's value

```
@property
def hour(self):
    """Return the hour."""
    return self._hour

@hour.setter
def hour(self, hour):
    """Set the hour."""
    if not (0 <= hour < 24):
        raise ValueError(f'Hour ({hour}) must be 0-23')

    self._hour = hour
```

Class Time: `hour` Read-Write Property (cont.)

- The `@property` decorator precedes the property's *getter* method, which receives only a `self` parameter
- A decorator adds code to the decorated function
 - Makes the `hour` function work with attribute syntax
- *getter* method's name is the property name
- A decorator of the form `@property_name.setter` (`@hour.setter`) precedes the property's *setter* method
- Method receives two parameters— `self` and a parameter (`hour`) representing the value being assigned to the property
- `__init__` invoked this `setter` to *validate* `__init__`'s `hour` argument *before* creating and initializing the object's `_hour` attribute
- A **read-write property** has both a *getter* and a *setter*
- A **read-only property** has only a *getter*

Class Time: `minute` and `second` Read-Write Properties

- The following methods named `minute` and `second` define read-write `minute` and `second` properties
- Each property's `setter` ensures that its second argument is in the range 0–59 (the valid range of values for minutes and seconds)

```
@property
def minute(self):
    """Return the minute."""
    return self._minute

@minute.setter
def minute(self, minute):
    """Set the minute."""
    if not (0 <= minute < 60):
        raise ValueError(f'Minute ({minute}) must be 0-59')

    self._minute = minute

@property
def second(self):
    """Return the second."""
    return self._second

@second.setter
def second(self, second):
    """Set the second."""
    if not (0 <= second < 60):
        raise ValueError(f'Second ({second}) must be 0-59')

    self._second = second
```

Class Time: Method `set_time`

- Method `set_time` changes *all three* attributes with a *single* method call
- The method uses the class's properties defined above

```
def set_time(self, hour=0, minute=0, second=0):  
    """Set values of hour, minute, and second."""  
    self.hour = hour  
    self.minute = minute  
    self.second = second
```

Class Time: Special Method `__repr__`

- When you pass an object to built-in function `repr`—which happens implicitly when you evaluate a variable in an IPython session—the corresponding class's `__repr__` **special method** is called to get a string representation of the object

```
def __repr__(self):  
    """Return Time string for repr()."""  
    return (f'Time(hour={self.hour}, minute={self.minute}, ' +  
            f'second={self.second})')
```

Class Time: Special Method `__repr__` (cont.)

- The Python documentation indicates that `__repr__` returns the “official” string representation of the object
- Should look like a constructor expression that creates and initializes the object

Class Time: Special Method `__str__`

- `__str__` special method is called implicitly when you
 - convert an object to a string with the built-in function `str`
 - `print` an object

```
def __str__(self):  
    """Print Time in 12-hour clock format."""  
    return (('12' if self.hour in (0, 12) else str(self.hour % 12)) +  
           f':{self.minute:0>2}:{self.second:0>2}' +  
           (' AM' if self.hour < 12 else ' PM'))
```

10.4.3 Class `Time` Definition Design Notes

Interface of a Class

- Class `Time`'s properties and methods define the class's **public interface**—
 - Properties and methods programmers should use to interact with objects of the class

Attributes Are Always Accessible

- Python does *not* prevent you from directly manipulating the data attributes `_hour`, `_minute` and `_second`

```
In [1]: from timewithproperties import Time
```

```
In [2]: wake_up = Time(hour=7, minute=45, second=30)
```

```
In [3]: wake_up._hour
```

```
Out[3]: 7
```

```
In [4]: wake_up._hour = 100
```

```
In [5]: wake_up
```

```
Out[5]: Time(hour=100, minute=45, second=30)
```

- After snippet [4], the `wake_up` object contains *invalid* data
- Python tutorial says, “**nothing in Python makes it possible to enforce data hiding—it is all based upon convention**”

10.6 Case Study: Card Shuffling and Dealing Simulation

- Class `Card` represents a playing card that has a face (`'Ace'` , `'2'` , `'3'` , ..., `'Jack'` , `'Queen'` , `'King'`) and a suit (`'Hearts'` , `'Diamonds'` , `'Clubs'` , `'Spades'`)
- Class `DeckOfCards` represents a deck of 52 playing cards as a list of `Card` objects

10.6.1 Test-Driving Classes `Card` and `DeckOfCards`

Before we look at classes `Card` and `DeckOfCards` , let's use an IPython session to demonstrate their capabilities.

Creating, Shuffling and Dealing the Cards

```
In [1]: from deck import DeckOfCards
```

```
In [2]: deck_of_cards = DeckOfCards()
```

- `DeckOfCards` method `__init__` creates the 52 `Card` objects in order by suit and by face within each suit
- Printing a `deck_of_cards` object calls its `__str__` method

```
In [3]: print(deck_of_cards)
```

Ace of Hearts	2 of Hearts	3 of Hearts	4 of Hearts
5 of Hearts	6 of Hearts	7 of Hearts	8 of Hearts
9 of Hearts	10 of Hearts	Jack of Hearts	Queen of Hearts
King of Hearts	Ace of Diamonds	2 of Diamonds	3 of Diamonds
4 of Diamonds	5 of Diamonds	6 of Diamonds	7 of Diamonds
8 of Diamonds	9 of Diamonds	10 of Diamonds	Jack of Diamonds
Queen of Diamonds	King of Diamonds	Ace of Clubs	2 of Clubs
3 of Clubs	4 of Clubs	5 of Clubs	6 of Clubs
7 of Clubs	8 of Clubs	9 of Clubs	10 of Clubs
Jack of Clubs	Queen of Clubs	King of Clubs	Ace of Spades
2 of Spades	3 of Spades	4 of Spades	5 of Spades
6 of Spades	7 of Spades	8 of Spades	9 of Spades
10 of Spades	Jack of Spades	Queen of Spades	King of Spades

```
In [4]: deck_of_cards.shuffle()
```

```
In [5]: print(deck_of_cards)
```

8 of Diamonds	8 of Clubs	7 of Clubs	Queen of Diamonds
10 of Spades	2 of Clubs	10 of Hearts	6 of Diamonds
5 of Hearts	Queen of Spades	7 of Hearts	5 of Clubs
9 of Hearts	King of Hearts	9 of Clubs	6 of Hearts
2 of Spades	10 of Diamonds	6 of Spades	5 of Spades
10 of Clubs	3 of Spades	Ace of Spades	2 of Diamonds
Ace of Diamonds	4 of Hearts	3 of Diamonds	8 of Hearts
Queen of Hearts	2 of Hearts	8 of Spades	Queen of Clubs
Jack of Hearts	3 of Clubs	Ace of Hearts	9 of Spades
7 of Diamonds	4 of Clubs	Jack of Spades	5 of Diamonds
4 of Diamonds	6 of Clubs	Ace of Clubs	King of Diamonds
3 of Hearts	7 of Spades	Jack of Diamonds	King of Spades
Jack of Clubs	King of Clubs	9 of Diamonds	4 of Spades

Dealing Cards

- Can deal one `Card` at a time by calling method `deal_card`
- IPython calls the returned `Card` object's `__repr__` method to produce the string output

```
In [6]: deck_of_cards.deal_card()
```

```
Out[6]: Card(face='8', suit='Diamonds')
```

Class `Card`'s Other Features

- Deal another card and pass it to the built-in `str` function

```
In [7]: card = deck_of_cards.deal_card()
```

```
In [8]: str(card)
```

```
Out[8]: '8 of Clubs'
```

- Each `Card` has a corresponding image file name, which you can get via the `image_name` read-only property
 - Used when we display the `Card`s as images

```
In [9]: card.image_name
```

```
27 Out[9]: '8_of_Clubs.png'
```

Card Class

```
# card.py
"""Card class that represents a playing card and its image file name."""

class Card:
    FACES = ['Ace', '2', '3', '4', '5', '6', '7', '8', '9', '10', 'Jack', 'Queen', 'King']
    SUITS = ['Hearts', 'Diamonds', 'Clubs', 'Spades']

    def __init__(self, face, suit):
        """Initialize a Card with a face and suit."""
        self._face = face
        self._suit = suit

    @property
    def face(self):
        """Return the Card's self._face value."""
        return self._face

    @property
    def suit(self):
        """Return the Card's self._suit value."""
        return self._suit
```

```
@property
def image_name(self):
    """Return the Card's image file name."""
    return str(self).replace(' ', '_') + '.png'

def __repr__(self):
    """Return string representation for repr()."""
    return f"Card(face='{self.face}', suit='{self.suit}')"

def __str__(self):
    """Return string representation for str()."""
    return f'{self.face} of {self.suit}'

def __format__(self, format):
    """Return formatted string representation."""
    return f'{str(self):{format}}'
```

DeckOfCards Class

```
# deck.py
"""Deck class represents a deck of cards."""
import random
from card import Card

class DeckOfCards:
    NUMBER_OF_CARDS = 52 # constant number of Cards

    def __init__(self):
        """Initialize the deck."""
        self._current_card = 0
        self._deck = []

        for count in range(DeckOfCards.NUMBER_OF_CARDS):
            self._deck.append(Card(Card.FACES[count % 13], Card.SUITS[count // 13]))
```

```

def shuffle(self):
    """Shuffle deck."""
    self._current_card = 0
    random.shuffle(self._deck)

def deal_card(self):
    """Return one Card."""
    try:
        card = self._deck[self._current_card]
        self._current_card += 1
        return card
    except:
        return None

def __str__(self):
    """Return a string representation of the current _deck."""
    s = ''

    for index, card in enumerate(self._deck):
        s += f'{self._deck[index]:<19}'
        if (index + 1) % 4 == 0:
            s += '\n'

    return s

```

10.6.2 Class `Card`—Introducing Class Attributes

Class Attributes `FACES` and `SUITS`

- Each object of a class has its own copies of the class's data attributes
- A **class attribute** (also called a **class variable**) represents *class-wide* information
 - Belongs to the *class*, not to a specific object of that class
- Two class attributes (lines 5–7):
 - `FACES` is a list of the card face names
 - `SUITS` is a list of the card suit names

```
# card.py
"""Card class that represents a playing card and its image file name."""

class Card:
    FACES = ['Ace', '2', '3', '4', '5', '6', '7', '8', '9', '10', 'Jack', 'Queen', 'King']
    SUITS = ['Hearts', 'Diamonds', 'Clubs', 'Spades']
```

Class Attributes `FACES` and `SUITS` (cont.)

- Define a class attribute by assigning a value to it inside the class's definition, but not inside any of the class's methods or properties (in which case, they'd be local variables)
- `FACES` and `SUITS` are *constants* that are not meant to be modified
- *Style Guide for Python Code* recommends naming your constants with all capital letters.

Class Attributes `FACES` and `SUITS` (cont.)

- We'll use `FACES` and `SUITS` to initialize each `Card` we create
- Do not need a separate copy of each list in every `Card` object
- Class attributes are typically accessed through the class's name (as in, `Card.FACES` or `Card.SUITS`)
- Class attributes exist as soon as you import their class's definition

Card Method `__init__`

- Method `__init__` defines a `Card`'s `_face` and `_suit` data attributes

```
def __init__(self, face, suit):  
    """Initialize a Card with a face and suit."""  
    self._face = face  
    self._suit = suit
```

Read-Only Properties `face`, `suit` and `image_name`

- Once a `Card` is created, its `face`, `suit` and `image_name` do not change, so these are read-only properties
- A property is not required to have a corresponding data attribute
- `Card` property `image_name`'s value is *created dynamically* by getting the `Card` object's string representation with `str(self)`, replacing any spaces with underscores and appending the `'.png'` filename extension

```
@property
def face(self):
    """Return the Card's self._face value."""
    return self._face

@property
def suit(self):
    """Return the Card's self._suit value."""
    return self._suit

@property
def image_name(self):
    """Return the Card's image file name."""
    return str(self).replace(' ', '_') + '.png'
```

Methods That Return String Representations of a Card

- `Card` provides three special methods that return string representations
- Method `__repr__` returns a string representation that looks like a constructor expression

```
def __repr__(self):  
    """Return string representation for repr()."""  
    return f"Card(face='{self.face}', suit='{self.suit}')
```

Methods That Return String Representations of a Card (cont.)

- Method `__str__` returns a string of the format `' face of suit '`

```
def __str__(self):  
    """Return string representation for str()."""  
    return f'{self.face} of {self.suit}'
```

Methods That Return String Representations of a Card (cont.)

- In the `__str__` method of class `DeckOfCards`, we use f-strings to format the `Card`s in fields of 19 characters each
- Class `Card`'s special method `__format__` is called when a `Card` object is *formatted* as a string

```
def __format__(self, format):  
    """Return formatted string representation for str()."""  
    return f'{str(self):{format}}'
```

- Second argument is the format string used to format the object
- To use the `format` parameter's value as a format specifier, enclose the parameter name in braces to the *right* of the colon

10.6.3 Class `DeckOfCards`

- Class attribute `NUMBER_OF_CARDS` represents the number of `Card`s in a deck
- Data attribute `_current_card` keeps track of which `Card` will be dealt next (0 – 51)
- Data attribute `_deck` is a list of 52 `Card` objects

Method `__init__`

- Initializes a `_deck` of `Card`s
- `for` statement fills the list `_deck` by appending new `Card` objects, each initialized with two strings—one from the list `Card.FACES` and one from `Card.SUITS`

```
# deck.py
"""Deck class represents a deck of Cards."""
import random
from card import Card

class DeckOfCards:
    NUMBER_OF_CARDS = 52 # constant number of Cards

    def __init__(self):
        """Initialize the deck."""
        self._current_card = 0
        self._deck = []

        for count in range(DeckOfCards.NUMBER_OF_CARDS):
            self._deck.append(Card(Card.FACES[count % 13], Card.SUITS[count // 13]))
```

Method `shuffle`

- Resets `_current_card` to 0, then shuffles the `Card`s in `_deck` using the `random` module's `shuffle` function

```
def shuffle(self):  
    """Shuffle deck."""  
    self._current_card = 0  
    random.shuffle(self._deck)
```

Method `deal_card`

- Deals one `Card` from `_deck`
- Returns `None` when there are no more `Card`s to deal

```
def deal_card(self):  
    """Return one Card."""  
    try:  
        card = self._deck[self._current_card]  
        self._current_card += 1  
        return card  
    except:  
        return None
```

Method `__str__`

- Returns a string representation of the deck in four columns with each `Card` left aligned in a field of 19 characters

```
def __str__(self):  
    """Return a string representation of the current _deck."""  
    s = ''  
  
    for index, card in enumerate(self._deck):  
        s += f'{self._deck[index]:<19}'  
        if (index + 1) % 4 == 0:  
            s += '\n'  
  
    return s
```

10.6.4 Displaying Card Images with Matplotlib

- Let's display card images
- We downloaded public-domain card images from Wikimedia Commons:

https://commons.wikimedia.org/wiki/Category:SVG_English_pattern_playing_cards

[Click link](#)

- Located in the `ch10` examples folder's `card_images` subfolder

```
In [1]: from deck import DeckOfCards
```

```
In [2]: deck_of_cards = DeckOfCards()
```

Enable Matplotlib in IPython

- `inline` required only in Jupyter

```
In [3]: %matplotlib inline
```

Create the Base Path for Each Image

- Use the `pathlib` module's `Path` class to construct the full path to each image on our system
- `Path` method `joinpath` appends the subfolder containing the card images to the path for the current folder (`.`)

```
In [4]: from pathlib import Path
```

```
In [5]: path = Path('.').joinpath('card_images')
```

Import the Matplotlib Features

- We'll use a function from `matplotlib.image` to load the images

```
In [6]: import matplotlib.pyplot as plt
```

```
In [7]: import matplotlib.image as mpimg
```

Create the `Figure` and `Axes` Objects

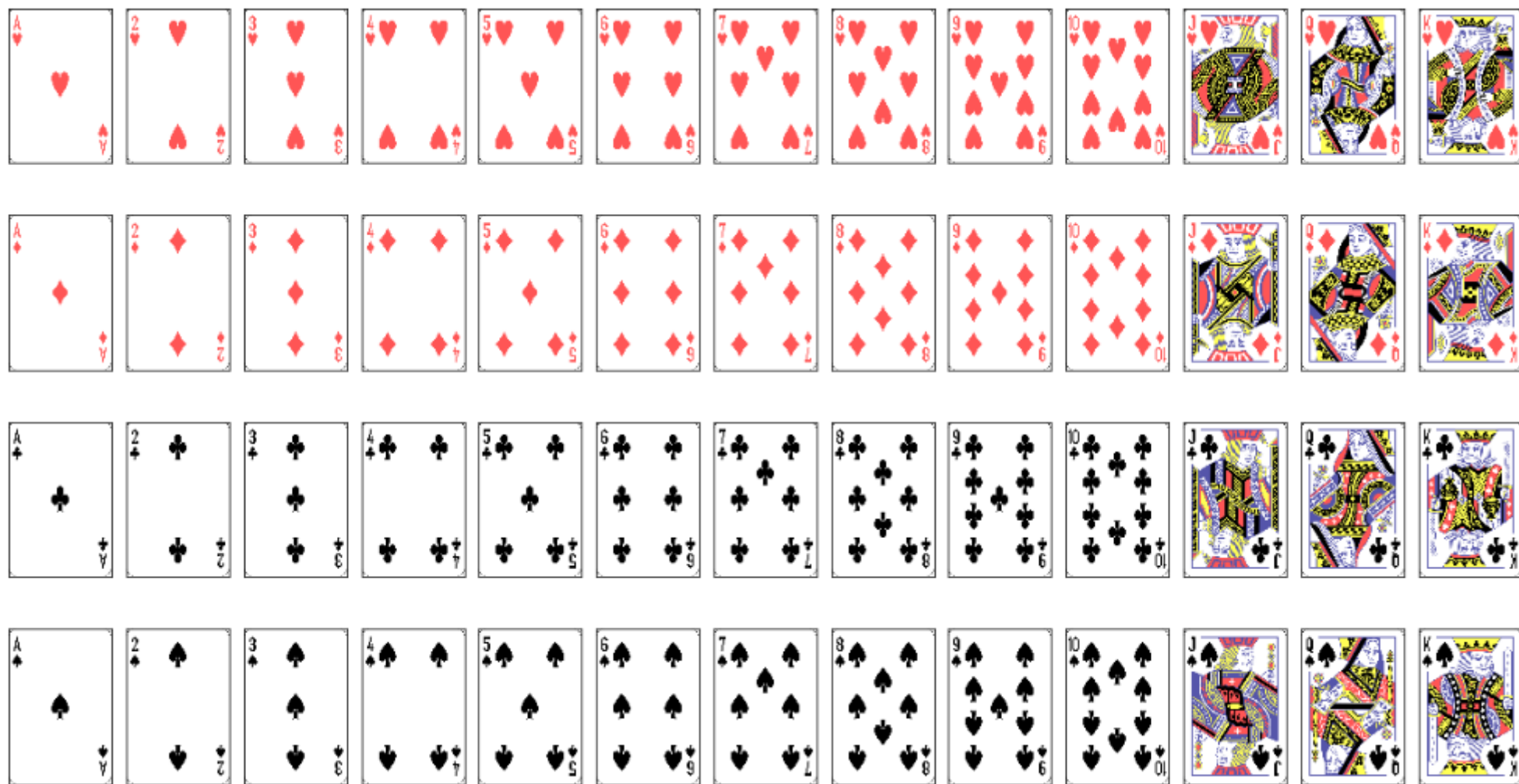
- **NOTE:** In Jupyter, all code that modifies the on-screen presentation of a `Figure` must be in one cell, so we combined several cells below
- The first statement in the following cell uses Matplotlib function `subplots` to create a `Figure` object in which we'll display the images as 52 *subplots* with four rows (`nrows`) and 13 columns (`ncols`)
- Returns a tuple containing the `Figure` and an array of the subplots' `Axes` objects

```
In [8]: figure, axes_list = plt.subplots(nrows=4, ncols=13)

# added next two statements to increase figure size in notebook
figure.set_figwidth(16)
figure.set_figheight(9)

for axes in axes_list.ravel():
    axes.get_xaxis().set_visible(False)
    axes.get_yaxis().set_visible(False)
    image_name = deck_of_cards.deal_card().image_name
    img = mpimg.imread(str(path.joinpath(image_name).resolve()))
    axes.imshow(img)

figure.tight_layout()
```



Configure the Axes Objects and Display the Images

- The loop iterates through all the Axes objects in `axes_list`
 - `ravel` provides a one-dimensional view of a multidimensional array
- For each Axes object,
 - hide the x- and y-axes.
 - deals a Card and get its `image_name`
 - get full path to the image
 - use `matplotlib.image` module's **`imread` function** to load the image
 - call Axes method **`imshow`** to display the current image in the current subplot

Maximize the Image Sizes

- Matplotlib Figure object's **`tight_layout` method** removes most of the extra white space in the window

Shuffle and Re-Deal the Deck

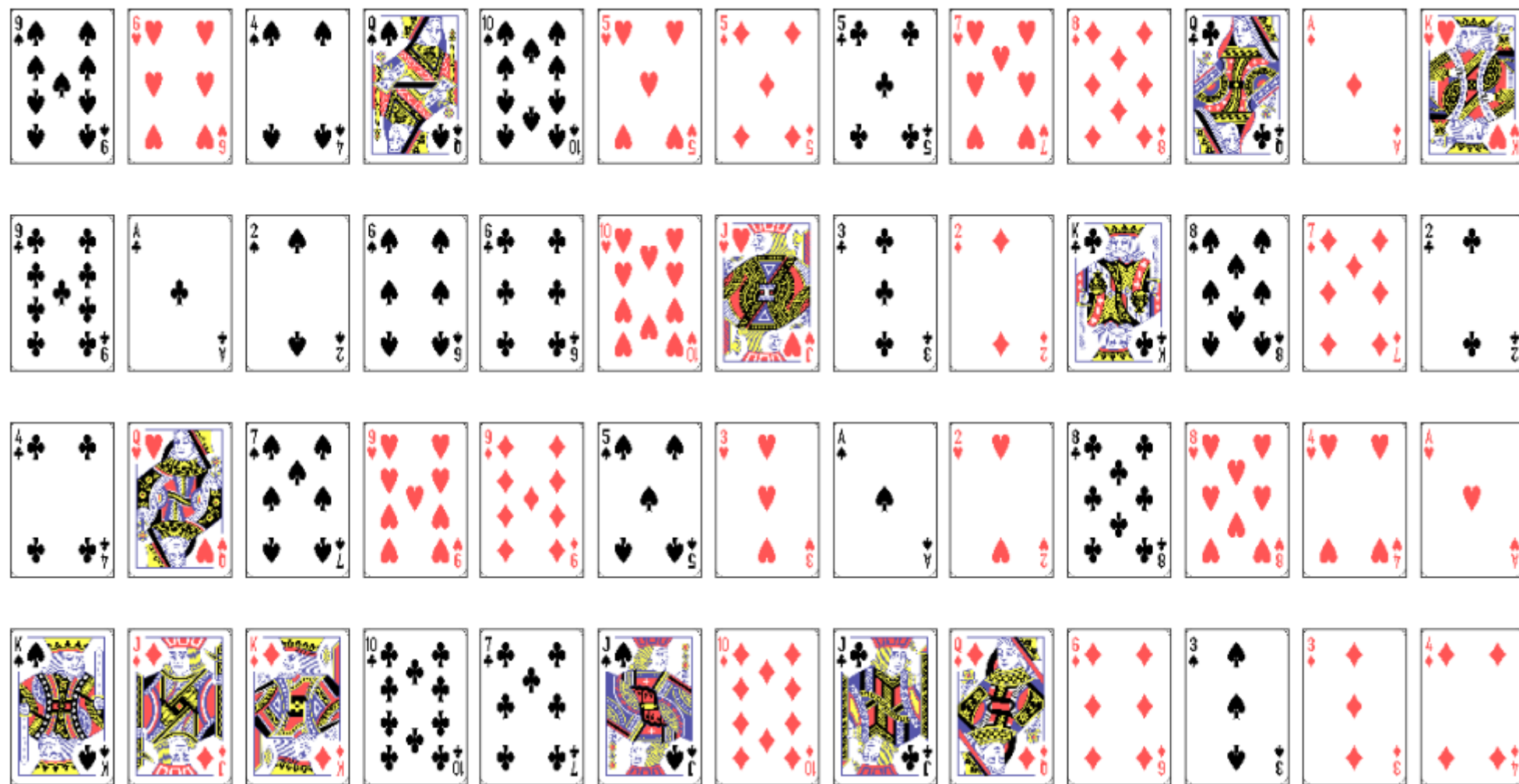
```
In [9]: deck_of_cards.shuffle()
```

```
In [10]: # added this statement to create a separate figure in the notebook
figure, axes_list = plt.subplots(nrows=4, ncols=13)

# added next two statements to increase figure size in notebook
figure.set_figwidth(16)
figure.set_figheight(9)

for axes in axes_list.ravel():
    axes.get_xaxis().set_visible(False)
    axes.get_yaxis().set_visible(False)
    image_name = deck_of_cards.deal_card().image_name
    img = mpimg.imread(str(path.joinpath(image_name).resolve()))
    axes.imshow(img)

# added this statement for execution in the notebook
figure.tight_layout()
```



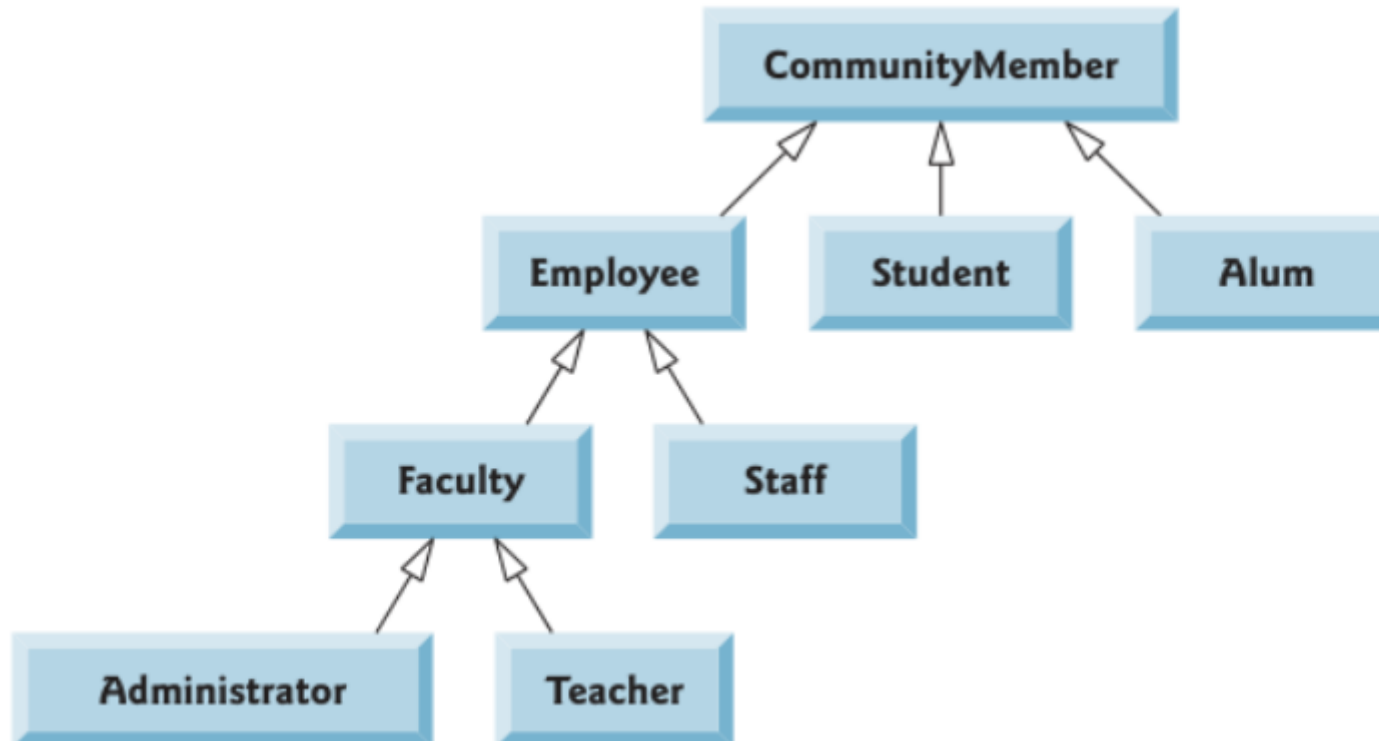
10.7 Inheritance: Base Classes and Subclasses

- Often, an object of one class is an object of another class as well. For example, a CarLoan is a Loan as are HomeImprovementLoans and MortgageLoans. Class CarLoan can be said to inherit from class Loan. In this context, class Loan is a base class, and class CarLoan is a subclass. A CarLoan is a specific type of Loan, but it's incorrect to claim that every Loan is a CarLoan—the Loan could be any type of loan. The following table lists several simple examples of base classes and subclasses—base classes tend to be “more general” and subclasses “more specific”:

Base class	Subclasses
Student	GraduateStudent, UndergraduateStudent
Shape	Circle, Triangle, Rectangle, Sphere, Cube
Loan	CarLoan, HomeImprovementLoan, MortgageLoan
Employee	Faculty, Staff
BankAccount	CheckingAccount, SavingsAccount

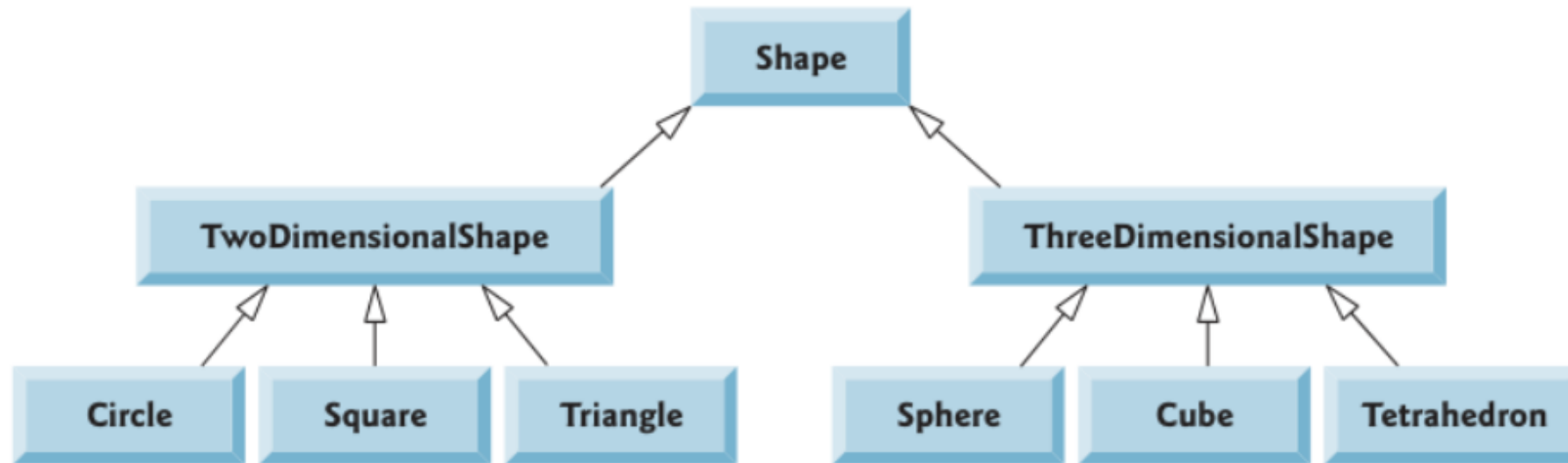
CommunityMember Inheritance Hierarchy

- Inheritance relationships form tree-like hierarchical structures. A base class exists in a hierarchical relationship with its subclasses. Let's develop a sample class hierarchy (shown in the following diagram), also called an **inheritance hierarchy**. A university community has thousands of members, including employees, students and alumni. Employees are either faculty or staff members. Faculty members are either administrators (e.g., deans and department chairpersons) or teachers. The hierarchy could contain many other classes. For example, students can be graduate or undergraduate students. Undergraduate students can be freshmen, sophomores, juniors or seniors. With **single inheritance**, a class is derived from one base class. With **multiple inheritance**, a subclass inherits from two or more base classes. Single inheritance is straightforward. Multiple inheritance is beyond the scope of this book. Before you use it, search online for the "diamond problem in Python multiple inheritance."



Shape Inheritance Hierarchy

- Now consider the Shape inheritance hierarchy in the following class diagram, which begins with base class Shape, followed by subclasses TwoDimensionalShape and ThreeDimensionalShape. Each Shape is either a TwoDimensionalShape or a ThreeDimensionalShape. The third level of this hierarchy contains specific types of TwoDimensionalShapes and ThreeDimensionalShapes. Again, we can follow the arrows from the bottom of the diagram to the topmost base class in this class hierarchy to identify several is-a relationships. For example, a Triangle is a TwoDimensionalShape and is a Shape, while a Sphere is a ThreeDimensionalShape and is a Shape. This hierarchy could contain many other classes. For example, ellipses and trapezoids also are TwoDimensionalShapes, and cones and cylinders also are ThreeDimensionalShapes.



“is a” vs. “has a”

- Inheritance produces **“is-a” relationships** in which an object of a subclass type may also be treated as an object of the base-class type. You’ve also seen “has-a” (composition) relationships in which a class has references to one or more objects of other classes as members.

10.8 Building an Inheritance Hierarchy; Introducing Polymorphism

- Hierarchy containing types of employees in a company's payroll app
- All employees of the company have a lot in common
 - *commission employees* (who will be represented as objects of a base class) are paid a percentage of their sales
 - *salaried commission employees* (who will be represented as objects of a subclass) receive a percentage of their sales *plus* a base salary

10.8.1 Base Class `CommissionEmployee`

Class `CommissionEmployee` provides the following features:

- Method `__init__` creates the data attributes `_first_name`, `_last_name` and `_ssn` (Social Security number), and uses the setter's of properties `gross_sales` and `commission_rate` to create their corresponding data attributes
- Read-only properties `first_name`, `last_name` and `ssn`, which return the corresponding data attributes
- Read-write properties `gross_sales` and `commission_rate` in which the setter's perform data validation
- Method `earnings`, which calculates and returns a `CommissionEmployee`'s earnings
- Method `__repr__`, which returns a string representation of a `CommissionEmployee`

```
# commmissionemployee.py
"""CommissionEmployee base class."""
from decimal import Decimal

class CommissionEmployee:
    """An employee who gets paid commission based on gross sales."""

    def __init__(self, first_name, last_name, ssn,
                 gross_sales, commission_rate):
        """Initialize CommissionEmployee's attributes."""
        self._first_name = first_name
        self._last_name = last_name
        self._ssn = ssn
        self.gross_sales = gross_sales # validate via property
        self.commission_rate = commission_rate # validate via property
```

```
@property
def first_name(self):
    return self._first_name

@property
def last_name(self):
    return self._last_name

@property
def ssn(self):
    return self._ssn

@property
def gross_sales(self):
    return self._gross_sales

@gross_sales.setter
def gross_sales(self, sales):
    """Set gross sales or raise ValueError if invalid."""
    if sales < Decimal('0.00'):
        raise ValueError('Gross sales must be >= to 0')

    self._gross_sales = sales
```

```

@property
def commission_rate(self):
    return self._commission_rate

@commission_rate.setter
def commission_rate(self, rate):
    """Set commission rate or raise ValueError if invalid."""
    if not (Decimal('0.0') < rate < Decimal('1.0')):
        raise ValueError(
            'Interest rate must be greater than 0 and less than 1')

    self._commission_rate = rate

def earnings(self):
    """Calculate earnings."""
    return self.gross_sales * self.commission_rate

def __repr__(self):
    """Return string representation for repr()."""
    return ('CommissionEmployee: ' +
        f'{self.first_name} {self.last_name}\n' +
        f'social security number: {self.ssn}\n' +
        f'gross sales: {self.gross_sales:.2f}\n' +
        f'commission rate: {self.commission_rate:.2f}')

```

All Classes Inherit Directly or Indirectly from Class `object`

- Every Python class inherits from an existing class
- When you do not explicitly specify the base class for a new class, Python assumes that the class inherits directly from class `object`
- Class `CommissionEmployee`'s header could have been written as

```
class CommissionEmployee(object):
```

- The parentheses after `CommissionEmployee` indicate inheritance and may contain
 - a single class for single inheritance
 - a comma-separated list of base classes for multiple inheritance

All Classes Inherit Directly or Indirectly from Class `object` (cont.)

- `CommissionEmployee` inherits all the methods of class `object`
- Two of the many methods inherited from `object` are `__repr__` and `__str__`
 - So *every* class has these methods that return string representations of the objects on which they're called
- When a base-class method implementation is inappropriate for a derived class, that method can be **overridden** (i.e., redefined) in the derived class with an appropriate implementation
 - Method `__repr__` overrides the default implementation from class `object`

Testing Class `CommissionEmployee`

- test some of `CommissionEmployee`'s features

```
In [1]: from commissionemployee import CommissionEmployee
```

```
In [2]: from decimal import Decimal
```

```
In [3]: c = CommissionEmployee('Sue', 'Jones', '333-33-3333',  
                                Decimal('10000.00'), Decimal('0.06'))
```

```
In [4]: c
```

```
Out[4]: CommissionEmployee: Sue Jones  
        social security number: 333-33-3333  
        gross sales: 10000.00  
        commission rate: 0.06
```

- calculate and display the `CommissionEmployee`'s earnings

```
In [5]: print(f'{c.earnings():,.2f}')
```

```
600.00
```

- change the `CommissionEmployee`'s gross sales and commission rate, then recalculate the earnings

```
In [6]: c.gross_sales = Decimal('20000.00')
```

```
In [7]: c.commission_rate = Decimal('0.1')
```

```
In [8]: print(f'{c.earnings():,.2f}')
```

```
2,000.00
```

10.8.2 Subclass `SalariedCommissionEmployee`

- With single inheritance, the subclass starts essentially the same as the base class
- The real strength of inheritance comes from the ability to define in the subclass additions, replacements or refinements for the features inherited from the base class.
- Many of a `SalariedCommissionEmployee`'s capabilities are similar, if not identical, to those of class `CommissionEmployee`
 - Both types of employees have first name, last name, Social Security number, gross sales and commission rate data attributes, and properties and methods to manipulate that data
- Inheritance enables us to “absorb” the features of a class *without* duplicating code

Declaring Class `SalariedCommissionEmployee`

- Subclass `SalariedCommissionEmployee` *inherits* most of its capabilities from class `CommissionEmployee`
- A `SalariedCommissionEmployee` *is a* `CommissionEmployee` (because inheritance passes on the capabilities of class `CommissionEmployee`)
- Class `SalariedCommissionEmployee` also has the following features:
 - Method `__init__`, which initializes all the data inherited from class `CommissionEmployee`, then uses the `base_salary` property's `setter` to create a `_base_salary` data attribute
 - Read-write property `base_salary`, in which the `setter` performs data validation.
 - A customized version of method `earnings`
 - A customized version of method `__repr__`

```
# salariedcommissionemployee.py
"""SalariedCommissionEmployee derived from CommissionEmployee."""
from commissionemployee import CommissionEmployee
from decimal import Decimal

class SalariedCommissionEmployee(CommissionEmployee):
    """An employee who gets paid a salary plus
    commission based on gross sales."""

    def __init__(self, first_name, last_name, ssn,
                  gross_sales, commission_rate, base_salary):
        """Initialize SalariedCommissionEmployee's attributes."""
        super().__init__(first_name, last_name, ssn,
                          gross_sales, commission_rate)
        self.base_salary = base_salary # validate via property
```

```

@property
def base_salary(self):
    return self._base_salary

@base_salary.setter
def base_salary(self, salary):
    """Set base salary or raise ValueError if invalid."""
    if salary < Decimal('0.00'):
        raise ValueError('Base salary must be >= to 0')

    self._base_salary = salary

def earnings(self):
    """Calculate earnings."""
    return super().earnings() + self.base_salary

def __repr__(self):
    """Return string representation for repr()."""
    return ('Salaried' + super().__repr__() +
            f'\nbase salary: {self.base_salary:.2f}')

```

Inheriting from Class `CommissionEmployee`

```
class SalariedCommissionEmployee(CommissionEmployee):
```

- specifies that class `SalariedCommissionEmployee` *inherits* from `CommissionEmployee`
- Don't see class `CommissionEmployee`'s data attributes, properties and methods in class `SalariedCommissionEmployee`, but they are there

Method `__init__` and Built-In Function `super`

- Each subclass `__init__` must explicitly call its base class's `__init__` to initialize the data attributes inherited from the base class
 - This call should be the first statement in the subclass's `__init__` method
- The notation `super().__init__` uses the built-in function `super` to locate and call the base class's `__init__` method

Overriding Method `earnings`

- Class `SalariedCommissionEmployee`'s `earnings` method overrides class `CommissionEmployee`'s `earnings` method to calculate the earnings of a `SalariedCommissionEmployee`
 - Obtains the portion of the earnings based on *commission alone* by calling `CommissionEmployee`'s `earnings` method with the expression `super().earnings()`

Overriding Method `__repr__`

- `SalariedCommissionEmployee`'s `__repr__` method overrides class `CommissionEmployee`'s `__repr__` method to return a `String` representation that's appropriate for a `SalariedCommissionEmployee`
- `super().__repr__()` calls `CommissionEmployee`'s `__repr__` method

Testing Class SalariedCommissionEmployee

```
In [9]: from salariedcommissionemployee import SalariedCommissionEmployee
```

```
In [10]: s = SalariedCommissionEmployee('Bob', 'Lewis', '444-44-4444',  
    Decimal('5000.00'), Decimal('0.04'), Decimal('300.00'))
```

```
In [11]: print(s.first_name, s.last_name, s.ssn, s.gross_sales,  
    s.commission_rate, s.base_salary)
```

Bob Lewis 444-44-4444 5000.00 0.04 300.00

- SalariedCommissionEmployee object has *all* of the properties of classes CommissionEmployee and SalariedCommissionEmployee
- Calculate and display the SalariedCommissionEmployee's earnings

```
In [12]: print(f'{s.earnings():.2f}')
```

500.00

- Modify the `gross_sales`, `commission_rate` and `base_salary` properties, then display the updated data via the `SalariedCommissionEmployee`'s `__repr__` method

```
In [13]: s.gross_sales = Decimal('10000.00')
```

```
In [14]: s.commission_rate = Decimal('0.05')
```

```
In [15]: s.base_salary = Decimal('1000.00')
```

```
In [16]: print(s)
```

```
SalariedCommissionEmployee: Bob Lewis  
social security number: 444-44-4444  
gross sales: 10000.00  
commission rate: 0.05  
base salary: 1000.00
```

- Calculate and display the `SalariedCommissionEmployee`'s updated earnings

```
In [17]: print(f'{s.earnings():,.2f}')
```

```
1,500.00
```

Testing the “is a” Relationship

Functions `issubclass` and `isinstance` are used to test “is a” relationships

- `issubclass` determines whether one class is derived from another

```
In [18]: issubclass(SalariedCommissionEmployee, CommissionEmployee)
```

```
Out[18]: True
```

- `isinstance` determines whether an object has an “is a” relationship with a specific type

```
In [19]: isinstance(s, CommissionEmployee)
```

```
Out[19]: True
```

```
In [20]: isinstance(s, SalariedCommissionEmployee)
```

```
Out[20]: True
```

10.8.3 Processing `CommissionEmployee`s and `SalariedCommissionEmployee`s Polymorphically

- With inheritance, every object of a subclass also may be treated as an object of that subclass's base class
- Can take advantage of this relationship to place objects related through inheritance into a list, then iterate through the list and treat each element as a base-class object
 - Allows a variety of objects to be processed in a *general* way

```
In [21]: employees = [c, s]
```

```
In [22]: for employee in employees:
          print(employee)
          print(f'{employee.earnings():,.2f}\n')
```

```
CommissionEmployee: Sue Jones
social security number: 333-33-3333
gross sales: 20000.00
commission rate: 0.10
2,000.00
```

```
SalariedCommissionEmployee: Bob Lewis
social security number: 444-44-4444
gross sales: 10000.00
commission rate: 0.05
base salary: 1000.00
1,500.00
```

- Correct string representation and earnings are displayed for each employee
- This is called *polymorphism*—a key capability of object-oriented programming (OOP)

10.9 Duck Typing and Polymorphism

- Most object-oriented programming languages require inheritance-based “is a” relationships to achieve polymorphic behavior
- Python also supports **duck typing**, which the Python documentation describes as:

A programming style which does not look at an object's type to determine if it has the right interface; instead, the method or attribute is simply called or used (“If it looks like a duck and quacks like a duck, it must be a duck.”).

- When processing an object at execution time, its type does not matter
- As long as the object has the data attribute, property or method (with the appropriate parameters) you wish to access, the code will work
- Reconsider the loop at the end of Section 10.8.3

```
for employee in employees:  
    print(employee)  
    print(f'{employee.earnings():,.2f}\n')
```

- Works properly as long as `employees` contains only objects that:
 - can be displayed with `print` (that is, they have a string representation)
 - have an `earnings` method which can be called with no arguments

- All classes inherit from `object` directly or indirectly, so they *all* inherit the default methods for obtaining string representations that print can display
- If a class has an `earnings` method that can be called with no arguments, we can include objects of that class in the list `employees`, even if the object's class does not have an "is a" relationship with class `CommissionEmployee`
- Consider class `WellPaidDuck`:

```
In [1]: class WellPaidDuck:
        def __repr__(self):
            return 'I am a well-paid duck'
        def earnings(self):
            return Decimal('1_000_000.00')
```

- Clearly not meant to be employees
- But, will work with the preceding loop

```
In [2]: from decimal import Decimal
```

```
In [3]: from commissionemployee import CommissionEmployee
```

```
In [4]: from salariedcommissionemployee import SalariedCommissionEmployee
```

```
In [5]: c = CommissionEmployee('Sue', 'Jones', '333-33-3333',  
                                Decimal('10000.00'), Decimal('0.06'))
```

```
In [6]: s = SalariedCommissionEmployee('Bob', 'Lewis', '444-44-4444',  
                                         Decimal('5000.00'), Decimal('0.04'), Decimal('300.00'))
```

```
In [7]: d = WellPaidDuck()
```

```
In [8]: employees = [c, s, d]
```

- Use duck typing to *polymorphically* process all three objects in the list

```
In [9]: for employee in employees:  
        print(employee)  
        print(f'{employee.earnings():,.2f}\n')
```

```
CommissionEmployee: Sue Jones  
social security number: 333-33-3333  
gross sales: 10000.00  
commission rate: 0.06  
600.00
```

```
SalariedCommissionEmployee: Bob Lewis  
social security number: 444-44-4444  
gross sales: 5000.00  
commission rate: 0.04  
base salary: 300.00  
500.00
```

```
I am a well-paid duck  
1,000,000.00
```

10.10 Operator Overloading

- Can use **operator overloading** to define how Python's operators should handle objects of your own types
- Can overload most operators
- For every overloadable operator, class `object` defines a special method
 - e.g., `__add__` for addition (`+`) or `__mul__` for multiplication (`*`)
- Overriding these methods enables you to define how a given operator works for objects of your custom class
- Complete list of special methods

<https://docs.python.org/3/reference/datamodel.html#special-method-names>

[Click link](#)

Operator Overloading Restrictions

- Precedence cannot be changed by overloading
- Left-to-right or right-to-left grouping of an operator cannot be changed
- “Arity” of an operator—whether it's unary or binary—cannot be changed
- Cannot create new operators
- How an operator works on objects of built-in types cannot be changed
- Works only with objects of custom classes or with a mixture of an object of a custom class and an object of a built-in type

Complex Numbers

- We'll define a class named `Complex` that represents complex numbers
- Complex numbers, like $-3 + 4i$ and $6.2 - 11.73i$, have the form

`realPart + imaginaryPart * i`

- `i` is the square root of -1
- We'll overload `+` and `+=`

10.10.1 Test-Driving Class `Complex`

```
In [1]: from complexnumber import Complex
```

- Create and display a couple of `Complex` objects

```
In [2]: x = Complex(real=2, imaginary=4)
```

```
In [3]: x
```

```
Out[3]: (2 + 4i)
```

```
In [4]: y = Complex(real=5, imaginary=-1)
```

```
In [5]: y
```

```
Out[5]: (5 - 1i)
```

- Use the `+` operator to add the `Complex` objects `x` and `y`
- Adds the real parts of the two operands and the imaginary parts of the two operands, then returns a new `Complex` object

```
In [6]: x + y
```

```
Out[6]: (7 + 3i)
```

- `+` does not modify either of its operands

```
In [7]: x
```

```
Out[7]: (2 + 4i)
```

```
In [8]: y
```

```
Out[8]: (5 - 1i)
```

- Use `+=` to add `y` to `x` and store the result in `x`
- `+=` operator *modifies* its left operand

```
In [9]: x += y
```

```
In [10]: x
```

```
Out[10]: (7 + 3i)
```

```
In [11]: y
```

```
Out[11]: (5 - 1i)
```

10.10.2 Class `Complex` Definition

Method `__init__`

- Initializes the `real` and `imaginary` data attributes

```
# complexnumber.py
"""Complex class with overloaded operators."""

class Complex:
    """Complex class that represents a complex number
    with real and imaginary parts."""

    def __init__(self, real, imaginary):
        """Initialize Complex class's attributes."""
        self.real = real
        self.imaginary = imaginary
```

Overloaded + Operator

- Overridden special method `__add__` defines how to overload the `+` operator

```
def __add__(self, right):  
    """Overrides the + operator."""  
    return Complex(self.real + right.real,  
                   self.imaginary + right.imaginary)
```

- Methods that overload binary operators must provide two parameters
 - the *first* (`self`) is the *left* operand
 - the *second* (`right`) is the *right* operand
- We do *not* modify the contents of either of the original operands
 - Matches our intuitive sense of how this operator should behave

Overloaded `+=` Augmented Assignment

- Override special method `__iadd__` to define how `+=` adds two `Complex` objects

```
def __iadd__(self, right):  
    """Overrides the += operator."""  
    self.real += right.real  
    self.imaginary += right.imaginary  
    return self
```

- Augmented assignments modify their left operands, so method `__iadd__` modifies the `self` object, which represents the left operand, then returns `self`

Method `__repr__`

```
def __repr__(self):  
    """Return string representation for repr()."""  
    return (f'({self.real} ' +  
            ('+' if self.imaginary >= 0 else '-') +  
            f' {abs(self.imaginary)}i)')
```

10.16 Intro to Data Science: Time Series and Simple Linear Regression

- **Time series:** Sequences of values (**observations**) associated with points in time
 - daily closing stock prices
 - hourly temperature readings
 - changing positions of a plane in flight
 - annual crop yields
 - quarterly company profits
 - time-stamped tweets from Twitter users worldwide
- We'll use simple linear regression to make predictions from time series data

Time Series

- **Univariate time series:** *One* observation per time
- **Multivariate time series:** *Two or more* observations per time
- Two tasks often performed with time series are:
 - **Time series analysis**, which looks at existing time series data for patterns (like **seasonality**), helping data analysts understand the data
 - **Time series forecasting**, which uses past data to predict the future
- We'll perform time series forecasting

Simple Linear Regression

- Given a collection of values representing an **independent variable** (the month/year combination) and a **dependent variable** (the average high temperature for that month/year), simple linear regression describes the relationship between these variables with a straight line, known as the **regression line**

Linear Relationships

- Given a Fahrenheit temperature, we can calculate the corresponding Celsius temperature using:

$$c = 5 / 9 * (f - 32)$$

- f (the Fahrenheit temperature) is the *independent variable*
- c (the Celsius temperature) is the *dependent variable*
- Each value of c *depends on* the value of f used in the calculation

Linear Relationships (cont.)

- Plotting Fahrenheit temperatures and their corresponding Celsius temperatures produces a straight line

```
In [1]: # enable high-res images in notebook
%config InlineBackend.figure_format = 'retina'
%matplotlib inline
def c(f):
    return 5 / 9 * (f - 32)
```

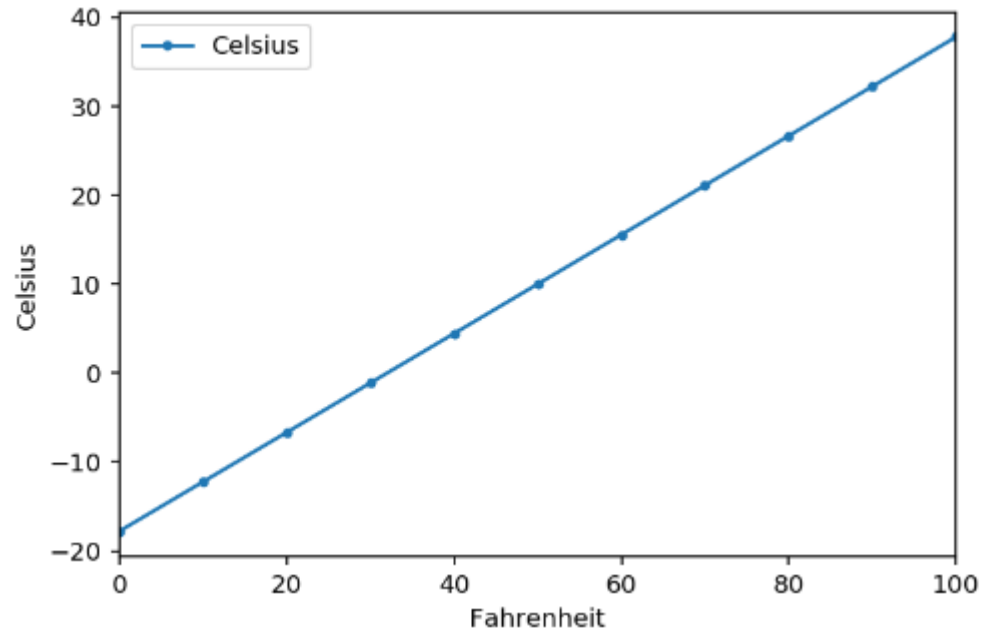
```
In [2]: temps = [(f, c(f)) for f in range(0, 101, 10)]
```

- Place the data in a `DataFrame`, then use its `plot` method to display the linear relationship between the temperatures
- `style` keyword argument controls the data's appearance
 - `'.-'` indicates that each point should appear as a dot, and that lines should connect the dots

```
In [3]: import pandas as pd
```

```
In [4]: temps_df = pd.DataFrame(temps, columns=['Fahrenheit', 'Celsius'])
```

```
In [5]: axes = temps_df.plot(x='Fahrenheit', y='Celsius', style='.-')
y_label = axes.set_ylabel('Celsius')
```



Components of the Simple Linear Regression Equation

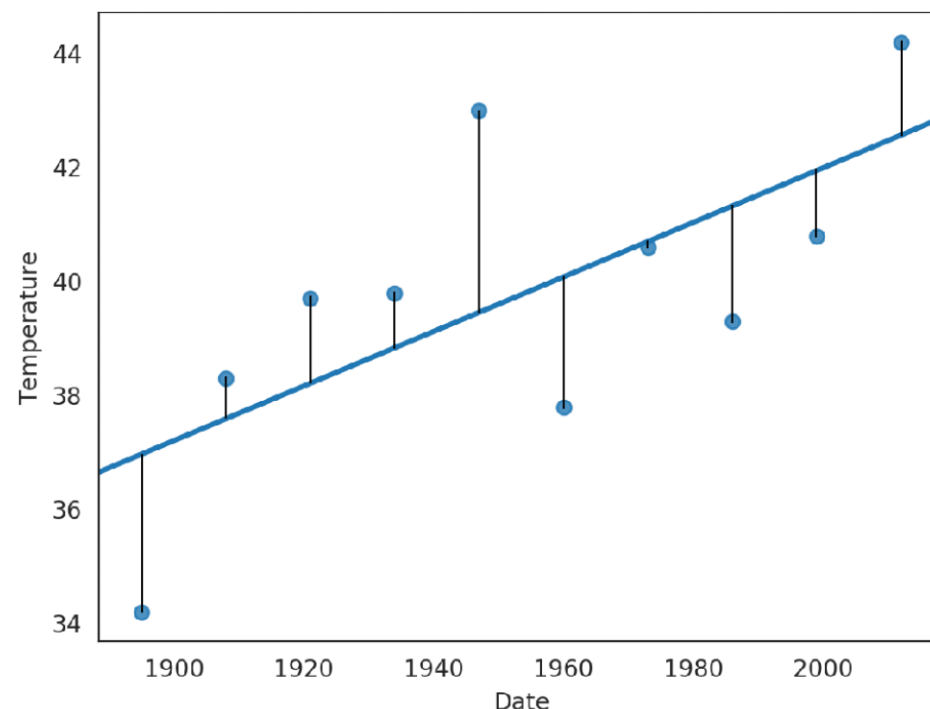
The points along any straight line can be calculated with:

$$y = mx + b$$

- ***m*** is the line's **slope**,
- ***b*** is the line's **intercept** with the **y-axis** (at ***x* = 0**),
- ***x*** is the independent variable (the date in this example)
- ***y*** is the dependent variable (the temperature in this example)
- In simple linear regression, ***y*** is the *predicted value* for a given ***x***

Function `linregress` from the SciPy's `stats` Module

- Simple linear regression determines slope (m) and intercept (b) of a straight line that best fits your data
- Following diagram shows a few of the time-series data points we'll process in this section and a corresponding regression line
 - We added vertical lines to indicate each data point's distance from the regression line
- Simple linear regression algorithm iteratively adjusts the slope and intercept and, for each adjustment, calculates the square of each point's distance from the line
- "Best fit" occurs when slope and intercept values minimize sum of those squared distances
 - **ordinary least squares** calculation
- **SciPy (Scientific Python)** is widely used for engineering, science and math in Python
 - `linregress` function (from the `scipy.stats` module) performs simple linear regression for you



Getting Weather Data from NOAA

- The National Oceanic and Atmospheric Administration (NOAA) offers public historical data including time series for average high temperatures in specific cities over various time intervals
- Obtained the January average high temperatures for New York City from 1895 through 2018 from NOAA's "Climate at a Glance" time series at:

<https://www.ncdc.noaa.gov/cag/>

[Click link](#)

- `ave_hi_nyc_jan_1895-2018.csv` in the `ch10` examples folder
- Three columns per observation:
 - `Date` —A value of the form `'YYYYMM'` (such as `'201801'`). `MM` is always `01` because we downloaded data for only January of each year.
 - `value` —A floating-point Fahrenheit temperature.
 - `Anomaly` —The difference between the value for the given date and average values for all dates (not used in this example)

Loading the Average High Temperatures into a DataFrame

```
In [6]: nyc = pd.read_csv('ave_hi_nyc_jan_1895-2018.csv')
```

- Get a sense of the data

```
In [7]: nyc.head()
```

Out[7]:

	Date	Value	Anomaly
0	189501	34.2	-3.2
1	189601	34.7	-2.7
2	189701	35.5	-1.9
3	189801	39.6	2.2
4	189901	36.4	-1.0

```
In [8]: nyc.tail()
```

Out[8]:

	Date	Value	Anomaly
119	201401	35.5	-1.9
120	201501	36.1	-1.3
121	201601	40.8	3.4
122	201701	42.8	5.4
123	201801	38.7	1.3

Cleaning the Data

- For readability, rename the 'Value' column as 'Temperature'

```
In [9]: nyc.columns = ['Date', 'Temperature', 'Anomaly']
```

```
In [10]: nyc.head(3)
```

```
Out[10]:
```

	Date	Temperature	Anomaly
0	189501	34.2	-3.2
1	189601	34.7	-2.7
2	189701	35.5	-1.9

- Seaborn labels the tick marks on the *x*-axis with Date values
- *x*-axis labels will be more readable if they do not contain 01 (for January), so we'll remove it from each Date
- Check the column's type:

```
In [11]: nyc.Date.dtype
```

```
Out[11]: dtype('int64')
```

- Values are integers, so we can divide by 100 to truncate the last two digits
- Series method floordiv performs *integer division* on every element of the Series

```
In [12]: nyc.Date = nyc.Date.floordiv(100)
```

```
In [13]: nyc.head(3)
```

```
Out[13]:
```

	Date	Temperature	Anomaly
0	1895	34.2	-3.2
1	1896	34.7	-2.7
2	1897	35.5	-1.9

Calculating Basic Descriptive Statistics for the Dataset

- Call `describe` on the `Temperature` column

```
In [14]: pd.set_option('precision', 2)
```

```
In [15]: nyc.Temperature.describe()
```

```
Out[15]: count    124.00  
mean      37.60  
std        4.54  
min       26.10  
25%       34.58  
50%       37.60  
75%       40.60  
max       47.60  
Name: Temperature, dtype: float64
```

Forecasting Future January Average High Temperatures

- **SciPy (Scientific Python) library** widely used for engineering, science and math in Python
- **stats module** provides function **linregress**, which calculates a regression line's *slope* and *intercept*

```
In [16]: from scipy import stats
```

```
In [17]: linear_regression = stats.linregress(x=nyc.Date,  
                                             y=nyc.Temperature)
```

- **linregress** receives two one-dimensional arrays of the same length representing the data points' **x**- and **y**-coordinates
 - **x** and **y** represent the independent and dependent variables, respectively
- Returns the regression line's **slope** and **intercept**

```
In [18]: linear_regression.slope
```

```
Out[18]: 0.014771361132966163
```

```
In [19]: linear_regression.intercept
```

```
Out[19]: 8.694993233674289
```

- Use these values with the simple linear regression equation for a straight line to predict the average January temperature in New York City for a given year
- In the following calculation, `linear_regression.slope` is m , `2019` is x (the date value for which you'd like to predict the temperature), and `linear_regression.intercept` is b :

```
In [20]: linear_regression.slope * 2019 + linear_regression.intercept
```

```
Out[20]: 38.51837136113297
```

- Approximate the average temperature for January of 1890:

```
In [21]: linear_regression.slope * 1890 + linear_regression.intercept
```

```
Out[21]: 36.612865774980335
```

- We had data for 1895–2018
- The further you go outside this range, the less reliable the predictions will be

Plotting the Average High Temperatures and a Regression Line

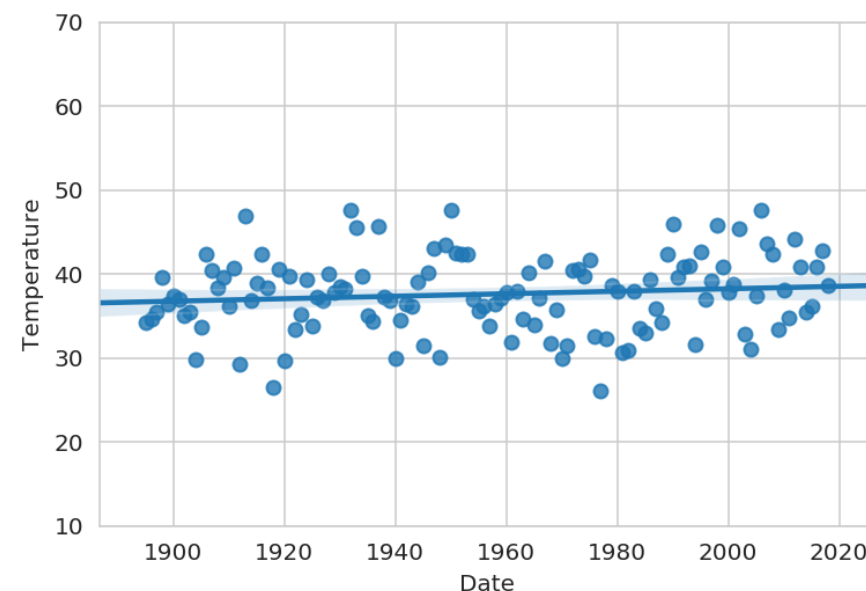
- Seaborn's `regplot` function plots each data point with the dates on the **x**-axis and the temperatures on the **y**-axis
- Creates a **scatter plot** or **scattergram** representing the `Temperature`s for the given `Date`s and adds the regression line
- Function `regplot`'s `x` and `y` keyword arguments are one-dimensional arrays of the same length representing the **x-y** coordinate pairs to plot

```
In [22]: import seaborn as sns
```

```
In [23]: sns.set_style('whitegrid')
```

```
In [24]: axes = sns.regplot(x=nyc.Date, y=nyc.Temperature)
         axes.set_ylim(10, 70)
```

```
Out[24]: (10, 70)
```



- In this graph, the y-axis represents a 21.5-degree temperature range between the minimum of 26.1 and the maximum of 47.6
- By default, the data appears to be spread significantly above and below the regression line, making it difficult to see the linear relationship
- Common issue in data analytics visualizations
- Seaborn and Matplotlib *auto-scale* the axes, based on the data's range of values
- We scaled the **y**-axis range of values to emphasize the linear relationship

Getting Time Series Datasets

Sources time-series dataset

<https://data.gov/>

[Click link](#)

This is the U.S. government's open data portal. Searching for "time series" yields over 7200 time-series datasets.

<https://www.ncdc.noaa.gov/cag/>

[Click link](#)

The National Oceanic and Atmospheric Administration (NOAA) Climate at a Glance portal provides both global and U.S. weather-related time series.

<https://www.esrl.noaa.gov/psd/data/timeseries/>

[Click link](#)

NOAA's Earth System Research Laboratory (ESRL) portal provides monthly and seasonal climate-related time series.

<https://www.quandl.com/search>

[Click link](#)

Quandl provides hundreds of free financial-related time series, as well as fee-based time series.

<https://datamarket.com/data/list/?q=provider:tsdl>

[Click link](#)

The Time Series Data Library (TSDL) provides links to hundreds of time series datasets across many industries.